

Furniture Modeling Language (FML)

Technical Specification & Syntax Guide for LLM Code Generation

Volney Torres Almendro

CNC & Parametric Design Automation Pipeline

June 12, 2026

Abstract

This document outlines the formal specification of the **Furniture Modeling Language (FML)**, a lightweight, indentation-driven declarative language designed to map complex cabinet layouts into parametric 3D models via FreeCAD. This specification serves as a standardized reference document and high-fidelity System Prompt template to instruct Large Language Models (LLMs) to output production-ready FML scripts without syntax regressions.

License & Ownership Notice

Author: Volney Torres Almendro

Copyright: © 2026 Volney Torres Almendro. All rights reserved.

License: This technical specification and its syntax architecture are protected under proprietary terms. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files to use, modify, and integrate the language parser into local automated environments (e.g., FreeCAD macro nodes), subject to maintaining this copyright notice in all copies or substantial portions of the software.

1 Introduction & Objective

The primary goal of FML is to bridge the gap between high-level macro requirements and the execution pipeline of a custom FreeCAD headless compiler (**FreeCADCmd**). By enforcing rigid indentation, zero-ambiguity geometry definitions, and isolated parameter blocks, FML allows complex multi-column, multi-row assemblies to be parsed deterministically into strict volumetric trees.

2 Global Initialization Parameters

Every compliant FML document must initialize the global boundary box and raw material attributes at the root of the file. These entries must contain no leading whitespace or indentation:

- **W:** Total module external width (*Width*) specified in millimeters.
- **H:** Total module external height (*Height*) specified in millimeters.
- **D:** Total module external depth (*Depth*) specified in millimeters.
- **T:** Nominal board raw material thickness (*Thickness*) specified in millimeters (e.g., 15 or 18).

3 Space Partitioning Logic (**split**)

The inner volume of an initialized module is subdivided using a top-down hierarchical tree framework governed by the **split** directive and an associated numerical array (**ratio**).

3.1 Partitioning Axes

- **split: horizontal:** Divides the parent container along the vertical axis, creating horizontal partitions (shelves, floors, or headers).
- **split: vertical:** Divides the parent container along the horizontal axis, creating vertical partitions (gables, dividers, or columns).

3.2 Proportions and Structural Indexing

- **ratio: [x, y, ...]:** An array of decimal floating-point numbers representing the proportional distribution of space. The total sum of all indices within the matrix must equate exactly to 1.0 (representing 100% of the available target domain).
- **Structural Space Naming:** Every sub-volume instantiated by a split sequence must be explicitly bound to a sequential index identifier: **Space_1:**, **Space_2:**, **Space_3:**, etc.
- **Indentation Framework:** Every lower hierarchical tier (child nodes or local properties) must feature an indentation increment of exactly **4 spaces** relative to its immediate parent node block.

4 Component Type Enums (**type**)

Inside a leaf container node (**Space_X:**), the physical state or mechanical treatment of the volume must be declared via the **type:** variable:

- **type: empty:** Instantiates an un-machined, open structural niche or pass-through bay. *Critical Operational Note:* This acts as a terminal execution node; it must never contain nested children or trailing properties.
- **type: doors:** Triggers the dynamic panel hardware rendering algorithm to compute and construct front door facades.
- **type: drawers:** Triggers the drawer box sub-assembly generation engine.

5 Precision Overrides & Hardware Parameters

When the active execution node type is explicitly set to **doors** or **drawers**, sub-assembly attributes must be passed to dictate manufacturing constraints. These parameter key-pairs must be placed at the **exact same indentation index** as the **type:** keyword itself.

6 Lexer Constraints & Compilation Safety

To ensure error-free compilation when running files through the automated **FreeCADCmd** parser node:

1. **Comment Tokenization:** Only the hash symbol (**#**) is recognized as a valid comment delimiter. Using alternate delimiters like **//**, **/***, or inline annotations will prevent proper tokenization and cause type-casting failures inside the float parser array.

Table 1: Dynamic Sub-Assembly Control Variables

Parameter Name	Functional Description	Standard Values / Formats
count	Quantity of modular panel elements to construct	1 (single slab) or X (split array)
door_style	Hardware assembly alignment paradigm	overlay (full lap) / inset (flush)
drawer_style	Drawer box facade positioning paradigm	overlay / inset
handle	Component index mapping for BOM generation	e.g., <code>perfil_aluminio</code>
gap	Global perimeter clearance offset metric	3.0 (Standard clearance required)
gap_top	Independent upper clearance offset override	Measurement in mm (e.g., 5.0)
gap_bottom	Independent lower clearance offset override	Measurement in mm
gap_left	Independent left clearance offset override	Measurement in mm
gap_right	Independent right clearance offset override	Measurement in mm

2. **Language Uniformity:** All structural and metadata documentation comments inside the file must be written in **English** to ensure clean processing across internationalized CAM engines.
3. **Key-Pair Separation:** Key-value pairs require an absolute colon assignment pattern followed by an explicit whitespace character (**key: value**).

7 Gold-Standard Code Implementation Reference

The following FML script represents a verified, error-free implementation of a complex structural layout. It defines a module with a lower split base (90cm/60cm arrangement where the left section features a 50% horizontal subdivision) and a perfectly aligned clean upper shelf alzada grid:

```

1  W: 1500
2  H: 2200
3  D: 460
4  T: 15
5
6  # Step 1: Execute main structural division (Base 70cm and Alzada 150cm)
7  split: horizontal
8  ratio: [0.31, 0.69]
9
10 # --- LOWER BASE STRUCTURAL SECTION (70 cm Height) ---
11 Space_1:
12     split: vertical
13     ratio: [0.6, 0.4] # 90cm left bay and 60cm right bay configuration
14
15     # Left Lower Bay: Features a 50% split horizontal division
16     Space_1:
17         split: horizontal
18         ratio: [0.5, 0.5]
19         Space_1:
20             type: empty
21         Space_2:
22             type: empty
23
24     # Right Lower Bay: Clean open structural niche
25     Space_2:
26         type: empty
27
28 # --- UPPER ALZADA SHELF SECTION (150 cm Height) ---
29 Space_2:
30     split: vertical
31     ratio: [0.6, 0.4] # Aligned 90cm left stack and 60cm right stack
32
33     # Left Alzada Column (90 cm width) - 4 Shelves array
34     Space_1:
35         split: horizontal
36         ratio: [0.5, 0.25, 0.25] # Heights: 75cm, 37.5cm, and 37.5cm
37
38         Space_1:
39             split: horizontal
40             ratio: [0.5, 0.5]
41             Space_1:
42                 type: empty
43             Space_2:
44                 type: empty
45         Space_2:
46             type: empty
47         Space_3:
48             type: empty
49
50     # Right Alzada Column (60 cm width) - 3 Shelves standard array
51     Space_2:
52         split: horizontal

```

```
53         ratio: [0.5, 0.25, 0.25] # Heights: 75cm, 37.5cm, and 37.5cm
54
55     Space_1:
56         type: empty
57     Space_2:
58         type: empty
59     Space_3:
60         type: empty
```

Listing 1: Production-ready FML reference file